GE Digital

bsfdeploy

8.0.8

User Guide

Copyright

Copyright 2021, General Electric Company and/or its affiliates ("GE"). All Rights Reserved. This document is the confidential and proprietary information of GE and may not be reproduced, transmitted, stored, or copied in whole or in part, or used to furnish information to others, without the prior written permission of GE.

Change History

Date	Change Description
20/08/21	Initial Version

Contents

1. Introduction **5**
1.1. Architecture 5
1.1.1. Components of bsfdeploy 6
1.1.1.1. Product Deployment Image (PDI) 6
1.1.1.2. bsfdeploy client 6
1.1.1.3. Input Manifest 6
2. Installation **7**
2.1. Install bsfdeploy 7
2.1.1. Prerequisites 7
2.1.2. Linux Installation 7
2.1.3. Windows Installation 7
3. Usage **8**
3.1. bsfdeploy Client Command 8
3.1.1. Common Options 8
3.2. Help Command 8
3.3. Usage Command 8
3.4. Actions 9
3.4.1. Plan 9
3.4.1.1. Options 9
3.4.2. Apply 10
3.4.2.1. Options 10
3.4.3. Status 11
3.4.3.1. Options 11
3.4.4. Destroy 11
3.4.4.1. Options 12
3.4.5. Test 12
3.4.5.1. Options 13
3.4.6. Taint 13
3.4.6.1. Options 13
3.4.7. Logs 14
3.4.8. Clean 14
3.4.8.1. Options 14
3.5. The Rendered Manifest 14
3.6. Interactive Mode 14
3.6.1. Multiple PDIs in Manifest 15
3.6.1.1. Example 15
3.7. Podman 16
4. Input Manifest **17**
4.1. Input Manifest Syntax 17
4.1.1. Top Level Syntax 17
4.1.2. Target Syntax 17
4.1.3. Product Syntax 18
4.1.4. pdi Syntax 18
4.1.5. param_group Syntax 19
4.1.6. Example Input Manifest 19

4.2. Example Manifest using dockerfile 20
4.3. Override Process 21
4.3.1. Inputs and Outputs of the Override Process 21
4.3.1.1. Default Manifest (Input) 21
4.3.1.2. Input Manifest (Input) 21
4.3.1.3. Rendered Manifest (Output) 21
4.3.2. Override Behaviour 22
4.3.2.1. Scalar 22
4.3.2.2. List 22
4.3.2.3. List item 22
4.3.2.4. Dictionary 23
4.3.3. Example 1 with Scalar Value 23
4.3.4. Example 2 with List, Dictionary, and missing parameters 24
4.4. Templating Process 25
4.4.1. Jinja2 Settings 25
4.4.1.1. Jinja2 env Filter 26
4.4.1.2. Jinja2 gethostbyname Filter 26
4.4.2. Processing 26
4.4.2.1. Example 1 - All in One 26
4.4.2.2. Example 2 - Using Template Inheritance and Import 29
4.5. Secret Management 30
4.5.1. Generation of the Keys 31
4.5.2. Encryption of Parameters 31
4.5.3. Decryption of the Parameters 31
4.5.4. Example 31

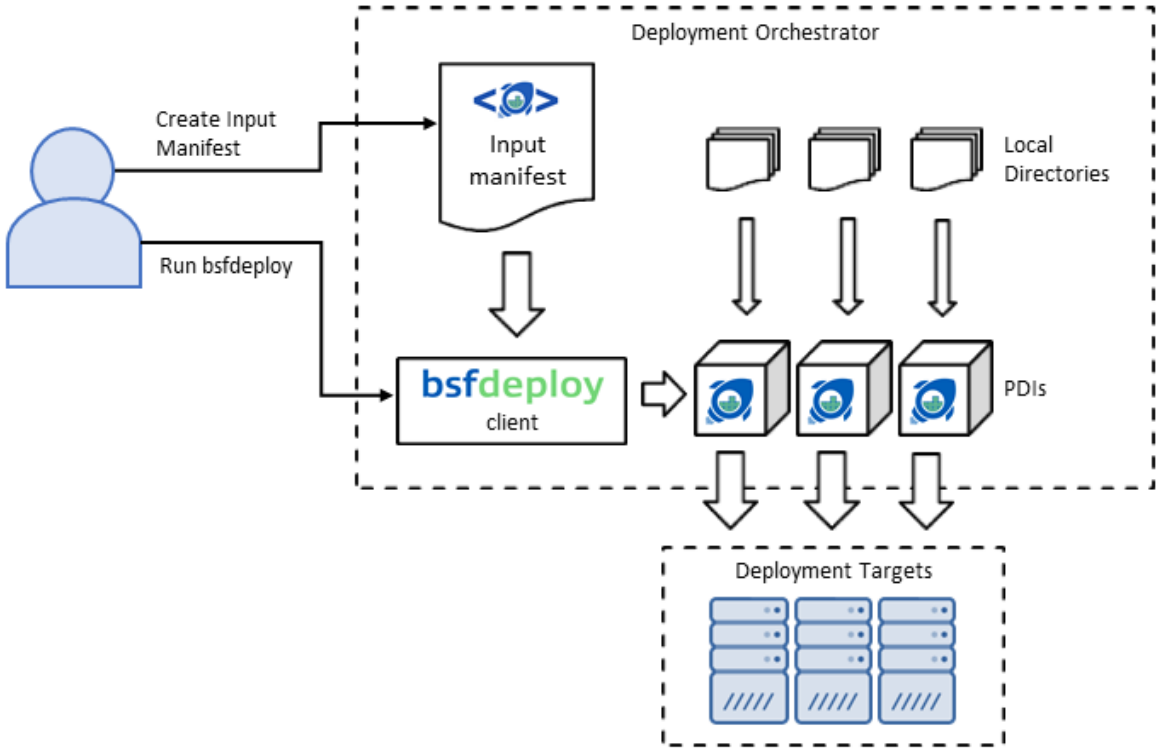
1. Introduction

`bsfdeploy` is an automated deployment engine used by GE Digital Grid. It enables fully automated push-button deployments, using a standard interface for parameterizing, deploying, smoke-testing, upgrading, rolling-back, and removing deployments. It can be used to deploy both the infrastructure and application layers, and is driven by a declarative manifest that can be stored in version control. Two major goals of `bsfdeploy` are:

1. To standardize the way all deployments are described and parameterized (via a common input manifest using a YAML syntax), executed and logged, whilst still giving the development team the ability to use the right tool for the job when it comes to deploying their particular application.
2. To simplify deployments, by avoiding the need to install or configure any specific deployment tools in advance - they will be baked into the PDI. The only prerequisite is Docker.

1.1. Architecture

`bsfdeploy` executes Product Deployment Images (PDIs). PDIs are effectively wrappers around the Product/Application installation automation. The `bsfdeploy` binary, is a means of running PDIs to deploy products to your target solution environment, which is capable of deploying individual products as well as complex solutions of products as defined by your Input Manifest



1.1.1. Components of bsfdeploy

1.1.1.1. Product Deployment Image (PDI)

This is a Docker image that encapsulates the deployment automation for the product or a custom, as well as any deployment tools that are required by the automation. Within each PDI exists the bsfdeploy engine which is the entry point of every PDI, and provides a consistent way of parameterizing and executing deployment automation.

1.1.1.2. bsfdeploy client

`bsfdeploy client` is an executable (available for both Linux & Windows) that can be run to apply the deployment of a solution consisting of one or more Products. It will automatically launch PDIs in the correct order, and with parameter overrides as defined by an Input Manifest.

How to use and execute `bsfdeploy` is explained in detail in Usage.

1.1.1.3. Input Manifest

The Input Manifest is passed as an input to the `bsfdeploy client`. It describes the desired state of a deployment in a declarative way. The manifest contains a list of products and their corresponding PDIs along with any parameters to be passed to the PDIs at deploy time.

Detailed specifications about the input manifest file are in Input Manifest.

2. Installation

2.1. Install bsfdeploy

The `bsfdeploy` client is available as an executable binary for both Linux and Windows.

2.1.1. Prerequisites

- Docker (`bsfdeploy` uses the environment's DOCKER_HOST to launch PDIs) OR
- Podman (again uses DOCKER_HOST - see Podman)
- Connectivity to a registry containing the PDIs
- Connectivity and credentials for `https://dig-grid-artifactory.apps.ge.com`

2.1.2. Linux Installation

The `bsfdeploy` executable binary is available in Digital Grid Artifactory, at https://dig-grid-artifactory.apps.ge.com/artifactory/bsf-generic-virtual/bsf/bsfdeploy. In this directory, versions of `bsfdeploy` are saved as tarballs named `bsfdeploy-<VERSION>.tgz`.

Download `bsfdeploy` from Build.GE Artifactory OR Digital Grid Artifactory, for example, using curl:

```
curl -u <user>:<password> \
-O "https://dig-grid-artifactory.apps.ge.com/artifactory/bsf-generic-virtual/bsf/bsfdeploy/bsfdeploy-<version>.tgz"

# Unzip the tar to get the `bsfdeploy` executable binary:
tar xvf bsfdeploy-<version>.tgz

# Move `bsfdeploy` (extracted from the tar) to /usr/local/bin:
mv ./bsfdeploy /usr/local/bin/bsfdeploy

# Cleanup the tarball:
rm bsfdeploy-<version>.tgz
```

2.1.3. Windows Installation

The `bsfdeploy` executable binary is available in Digital Grid Artifactory, at https://dig-grid-artifactory.apps.ge.com/artifactory/bsf-generic-virtual/bsf/bsfdeploy. In this directory, versions of the solution engine binary are saved as a zip named `bsfdeploy-<VERSION>.zip`.

Download and unzip the file to a windows executable called `bsfdeploy.exe`. Add the directory in which you save the executable to your PATH environment variable, so that you can call `bsfdeploy` from any location.

3. Usage

3.1. bsfdeploy Client Command

```
bsfdeploy [-h] [-a <ACTION>] [-u]
```

For windows, use `bsfdeploy.exe` instead of `bsfdeploy`.

3.1.1. Common Options

The options listed here have the same behaviour regardless of which action they are passed to.

Name, Shorthand	Description
`--target, -t`	When the target parameter is used, only the specified targets_id from Input Manifest will be taken into account. This parameter can be 1 target or a list of targets separated by , (ie. 'TargetX, TargetY')
`--product, -p`	When the product parameter is used, only the specified product_id from Input Manifest will be taken into account. You can specify both the product and target to work with one product from one target.
`--logfile, -l`	Destination file for the solution logs. Log will be appended to these files at each deployment. Default file names are 'solution.log' and 'solution.json'. Specify the name for the text log, and a json version will also appear with a '.json' extension
`--input-manifest, -i`	Specify the path/name of the solution input manifest file. Default location is `./manifest.yaml`

3.2. Help Command

```
bsfdeploy -h
```

The help command will display the help information for bsfdeploy. This will contain a full list of all the options that bsfdeploy supports.

3.3. Usage Command

```
bsfdeploy -u [-i INPUT_MANIFEST] [-t TARGET] [-p PRODUCT]
```

The usage command will display the usage information for a PDI as specified by the PDI's `usage_cmd` command.

Print Usage Information

```
bsfdeploy -u
```

3.4. Actions

3.4.1. Plan

```
bsfdeploy --action plan [-i INPUT MANIFEST] [-t TARGET] [-p PRODUCT] [-l LOGFILE]
          [-k KEYPATH] [-v] [--interactive] [--dns DNS SERVERS]
```

The plan command is used to create an execution plan for the deployment. This command is a convenient way to check whether the execution plan for your input manifest matches your expectations without making any changes to the state.

This will return the result of the evaluation which can either be `UPTODATE` or `REDEPLOY`. The rational for these are listed below

- UPTODATE: the product has been successfully deployed of the target with the same parameters
- REDEPLOY: the product need to be redeployed on the target because:
  - The parameters have changed since the last successful execution
  - The PDI image has changed since the last successful execution
  - The deployment is tainted
  - The last deployment ended in error
  - The product has not been deployed yet
  - An internal error occurred during the last deployment of the product

3.4.1.1. Options

Name, Shorthand	Description
`--encryption-key, -k`	When secret parameters are used, KEYPATH is pointing to the private key file.
`--verbose, -v`	Additionally returns the actual parameters used for the evaluation in yaml format, known as the rendered manifest.
`--interactive`	Will be passed to a called PDI. Allows interactive debugging of a PDI with the same environment during the run_cmd. See Interactive Mode.
`--dns`	Pass a list of DNS servers to the PDI as a docker parameter `--dns` to allow the use of dns server not set in the host within the PDI. Each DNS server element should be separated by ';'

Plan a specific product

```
bsfdeploy -a plan -t <targets_id> -p <product_id>
```

View rendered manifest

```
bsfdeploy -a plan -v
```

3.4.2. Apply

```
bsfdeploy --action apply [-i INPUT MANIFEST] [-t TARGET] [-p PRODUCT] [-l LOGFILE]
          [-k KEYPATH] [-v] [-f] [--parallel PARALLEL] [--interactive]
          [--dns DNS SERVERS]
```

The apply action is used to execute a deployment. This is done by reading the input manifest and then calculating the dependency order that the PDIs should be executed in. It will then do an apply action on each PDI to apply the changes required to reach the desired state that is specified by the input manifest

> bsfdeploy will only execute the apply action for the PDI if the plan returns `REDEPLOY` (unless the `--force` option is used).

3.4.2.1. Options

Name, Shorthand	Description
`--encryption-key, -k `	When secret parameters are used, KEYPATH is pointing to the private key file.
`--verbose, -v`	Return the output of the verbose plan action, then, if the product needs to be redeployed, will ouput the `run_cmd` execution and the verbose status of the deployment
`--force, -f`	Force the deployment of the selected target product regardless of the state of the deployment
`--parallel`	Allows parallel execution of PDIs for apply, test and destroy actions(Default is not parallel). Set an optional integer >1. It specifies a limit to the number of concurrent executions. If a value is not specified there is no limit on the number.
`--interactive`	Will be passed to called PDI. Allows interactive debugging of a PDI with the same environment during the run_cmd. See Interactive Mode.
`--dns`	Pass a list of DNS servers to the PDI as a docker parameter `--dns` to allow the use of dns server not set in the host within the PDI. Each DNS server element should be separated by ';'

To force the deployment of the manifest regardless of it's state, enter:

```
bsfdeploy -a apply --force
```

To apply a specific product from the input manifest:

```
bsfdeploy -a apply -t <target_name> -p <product_name>
```

Pass in a list of DNS servers

```
bsfdeploy -a apply --dns server1;server2
```

3.4.3. Status

```
bsfdeploy --action status [-i INPUT MANIFEST] [-t TARGET] [-p PRODUCT] [-l LOGFILE]
          [-v] [--interactive]
```

Return the status of the each of the products specified in the input manifest.

The available values for status are the following:

- `none`: the product has never been deployed on the target
- `success`: the product has been successfully deployed on the target
- `tainted`: the deployment of the product on this target has been labelled as taint to force the re-deployment during the next apply
- `error`: the last execution of the deployment ended in error.
- `inprogress`: deployment of the product is currently in progress
- `destroyed`: the deployment has been destroyed
- `destroy_error`: the execution of the last destroy ended in an error
- `destroyinprogress`: Destroy in progress. This state can persist if the execution has stopped during destroy automation

3.4.3.1. Options

Name, Shorthand	Description
`--verbose, -v`	Return the full state of the last deployment in a YAML format. In addition to what is returned in a non-verbose case, it shows the actual parameters used during the last deployment.
`--interactive`	Will be passed to called PDI. Allows interactive debugging of a PDI with the same environment during the run_cmd. See Interactive Mode

To view the status of a specific target/product:

```
bsfdeploy -a status -t <target_id> -p <product_id>
```

To use a specific input manifest:

```
bsfdeploy -a status -i <path_to_manifest>
```

3.4.4. Destroy

```
bsfdeploy --action destroy [-i INPUT MANIFEST] [-t TARGET] [-p PRODUCT] [-l LOGFILE]
          [-k KEYPATH] [--current] [--interactive] [--dns DNS SERVERS]
```

The destroy action is used to remove a deployment.

When you execute a destroy, you want to destroy what is currently deployed, rather than what you are about to deploy. For this reason, the default behaviour of the destroy command is to use the manifest that was supplied for the previous deployment when evaluating what to destroy. The manifest of the previous deployment is grabbed from the deployment's state. To override this behaviour, and destroy using the current manifest, use the `--current` flag.

If secret parameters were used in the previous deployment, the same encryption key that was used needs to be supplied to the destroy command through the `--encryption-key` option.

After a successful destroy execution , the status of the deployment is set to `destroyed`. If the execution is not successful, the status of the deployment is set to `destroy_error`

3.4.4.1. Options

Name, Shorthand	Description
`--encryption-key,-k`	When secret parameters are used, KEYPATH is pointing to the private key file.
`--current`	The destroy will take its parameters from the default/input manifest and local files.
`--interactive`	Will be passed to called PDI. Allows interactive debugging of a PDI with the same environment during the run_cmd. See Interactive Mode
`--dns`	Pass a list of DNS servers to the PDI as a docker parameter `--dns` to allow the use of dns server not set in the host within the PDI. Each DNS server element should be separated by ';'

To destroy the previous deployment from state that has secret text:

```
bsfdeploy -a destroy -k <path_to_encryption_key>
```

To destroy a solution specified by the current manifest rather than state:

```
bsfdeploy -a destroy --current
```

3.4.5. Test

```
bsfdeploy --action test [-i INPUT MANIFEST] [-t TARGET] [-p PRODUCT] [-l LOGFILE]
          [-k KEYPATH] [--current] [--interactive] [--dns DNS SERVERS]
```

The test action is used to run deployment validation tests (dvt). At the end of a test run a directory called 'results' will appear in the current working directory with the results of the tests. The 'results' directory will contain sub-directories for each target and product: `./results/<target>/<product>/myResults.xml`. The test action can only be invoked against an existing deployment, unless the `--current` option was passed. The output of the test action is displayed and logged. The test will take its parameters from the state file when not using the `--current` option. If a PDI does not support the test action then the PDI run ends in an error. If secret parameters were used in the former deployment, the encryption key needs to be provided.

State of of the deployment has a field `test_status` which can have the following value:

- `not_tested`: never tested
- `passed`: test passed
- `failed`: test failed
- `error`: The test automation had an error
- `inprogress`: Test in progress. This state can stay persist if PDI has been stopped during test automation

3.4.5.1. Options

Name, Shorthand	Description
`--encryption-key, -k`	When secret parameters are used, KEYPATH is pointing to the private key file.
`--current`	The test will take its parameters from the input manifest. The test will always execute, whatever the deployment status is, even if it was never deployed.
`--interactive`	Will be passed to called PDI. Allows interactive debugging of a PDI with the same environment during the run_cmd. See Interactive Mode.
`--dns`	Pass a list of DNS servers to the PDI as a docker parameter `--dns` to allow the use of dns server not set in the host within the PDI. Each DNS server element should be separated by ';'

To test the previous deployment from state that has secret text

```
bsfdeploy -a test -k <path_to_encryption_key>
```

To test solution specified by the current manifest rather than state

```
bsfdeploy -a test --current
```

3.4.6. Taint

```
bsfdeploy --action taint [-i INPUT MANIFEST] [-t TARGET] [-p PRODUCT] [-l LOGFILE]
                         [-v]
```

The taint action will set the status deployment as taint, to force the re-deployment during the next execution of bsfdeploy.

3.4.6.1. Options

Name, Shorthand	Description
`--verbose, -v`	Return the success of the action and the full state of the last deployment in a yaml format

To taint a specific product:

```
bsfdeploy -a taint -t <target_id> -p <product_id>
```

3.4.7. Logs

```
bsfdeploy --action logs [-i INPUT MANIFEST] [-t TARGET] [-p PRODUCT] [-l LOGFILE]
```

The logs actions will read the logs files of the deployment and print it to the console.

The logs action will print the log file of the deployment:

```
bsfdeploy -a logs
```

3.4.8. Clean

```
bsfdeploy --action clean [-i INPUT MANIFEST] [-t TARGET] [-p PRODUCT] [-l LOGFILE] [-q]
```

The clean action will delete the state and log files of the deployment.

3.4.8.1. Options

Name, Shorthand	Description
`--quiet, -q`	Will prevent the user confirmation from happening.

3.5. The Rendered Manifest

When the deployment engine runs, it passes the input manifest and the default manifest to the PDI and processes them, this results in the deployment engine creating a `rendered manifest`, which is the result of the override process of the default manifest by the relevant section of the input manifest.

It is possible to view what the rendered manifest will be using an `action: plan` combined with the `--verbose`/`-v` flag.

3.6. Interactive Mode

The `--interactive` flag will allow the ability to interactively debug a PDI. After a PDI executes its run_cmd, it will leave an open command prompt. This will have the same environment as the run_cmd, along with all output files generated by interpreters.

The `--interactive` flag can be used with the following actions:

- plan
- apply
- test
- status

- destroy

> If using `action: plan` and the `--force` flag is passed then this will **not** allow interactive debugging.

3.6.1. Multiple PDIs in Manifest

If there are multiple PDIs in the input manifest, then the `--target` and `--product` flag should be provided to specify which PDI the debugging should occur for. This has the limitation that the deployment of the PDIs must be done one by one.

3.6.1.1. Example

With the below input manifest:

```yaml
api_version: '5.0'
id: Solution_test
targets:
- id: TargetA
  products:
  - id: test-product1
    pdi:
      image: test-deploy1
      tag: bsfdeploytest
  - id: test-product2
    pdi:
      image: test-deploy2
      tag: bsfdeploytest
  - id: test-product3
    pdi:
      image: test-deploy3
      tag: test
  - id: test-product4
    pdi:
      image: test-deploy4
      tag: test
version: '1.0'
```

And we executed the deployment with:

```
bsfdeploy -a apply --interactive
```

We would get the below result:

```
[Solution Engine] Please limit to one PDI with target and product flags
[Solution Engine] when using interactive mode
[Solution Engine] List of PDIs
[Solution Engine] TargetA test-product1
[Solution Engine] TargetA test-product2
[Solution Engine] TargetA test-product3
[Solution Engine] TargetA test-product4
[Solution Engine] Stopped solution in error, Exception:  PDI Exception
[Solution Engine] EXECUTION PLAN ERROR: Limit interactive to one PDI
```

Instead, a PDI needs to be specified using the `--target` and `--product` flags. As noted above, this has the limitation of having to deploy the PDIs one by one due to the nature of the `--target` and `--product` flags.

```
bsfdeploy -a apply --target TargetA --product test-product1 --interactive
```

3.7. Podman

BSFDeploy uses the host systems's Container Engine, which can be either Docker or Podman. BSFDeploy communicates with the Container Engine through its REST API, through a socket. Whereas Docker always has a socket, for Podman you must explicotly start one, as per the api documentation.

```
podman system service -t 0 &
```

Once the socket is open point the DOCKER_HOST environment variable at the socket, and use BSFDeploy as you would with Docker:

```
export DOCKER_HOST=unix:/var/run/podman/podman.sock
```

4. Input Manifest

The Input Manifest is passed as an input to the `bsfdeploy client`. It describes the desired state of a deployment in a declarative way. The manifest contains a list of products and their corresponding PDIs along with any parameters to be passed to the PDIs at deploy time.

4.1. Input Manifest Syntax

4.1.1. Top Level Syntax

Key	Required?	Value Type	Notes
id	Yes	Scalar (string)	This ID MUST be unique for the solution that the manifest represents, as the state of the deployment is managed based on this value.
version	No	Scalar (string)	The `version` key allows the PDI end user to record a version of the manifest. It is not required and not currently used by the `deployment engine`.
api_version	Yes	Scalar (string)	The `api_version` refers to the syntax of the manifest, the `deployment engine` will raise a warning if the API version specified in the manifest is not compatible with the API version of itself. This version is the same as the pdi-base base image version that the PDI is built from, e.g. 5.0
targets	Yes	List (target)	The `target` is a way to group multiple product deployments to a target. A target could be an individual server, a cluster, or a group of servers. Targets is a list of individual targets. The `target`'s as part of the list are processed by the `deployment engine` one at a time.

4.1.2. Target Syntax

Key	Required?	Value Type	Notes
id	Yes	Scalar (string)	The target id MUST be unique within the list of targets.
serial	No	Boolean	Default is False. If a target has the serial flag, products from this target won't be deployed in parallel.
products	Yes	List (product)	List of type `product`. A product refers to an individual Product Deploy Image (PDI). Each product within the `products` list MUST be unique, i.e. you cannot list the same product twice for a single target. To follow good practice, the `id` of each product MUST match the `product_id` field of the PDI's `default_manifest.yaml`. NOTE: In exceptional cases only, primarily to provide a temporary workaround to early issues with the design and development of certain PDIs, it is possible to disable the requirement for the product to match the `product_id` field in the default manifest. Do this by prefixing the product with an '_'. This makes it possible to call the same PDI multiple times for the same target. For this feature to be employed, the PDI must be based on a deployment engine version >= 5.4.0. All future PDI development should be working to remove the need for this feature, with a view to its eventual deprecation.

4.1.3. Product Syntax

Key	Required?	Value Type	Notes
id	Yes	Scalar (string)	The product ID MUST match the `product_id` specified in the `default_manifest.yaml` of the PDI
pdi	Yes	Dict	Contains the image and tag of the PDI for this product
local_dir_mount_path	No	Scalar (string)	The `local_dir_mount_path` key points to the host folder to be mounted as /data/local within the PDI to be used to get external files from the PDI.
requires	No	List (Dict)	`requires` is used to specify the order of product deployment within a solution. Each list item is a dictionary which can contain a `target` key and a `product` key, the values of which respectively refer to target `id` and product `id` of the product with which this product requires to be deployed first. If the `target` key is not specified, the current target id is used.
depends_on	No	List (Dict)	`depends_on` is a superset of the `requires` key. In addition to specifying deployment order, it is used to specify that the current product must be re-deployed if the product being depended on is re-deployed. Each list item is a dictionary which can contain a `target` key and `product` key, the values of which respectively refer to target `id` and product `id` of the product with which this product is dependent upon. If the `target` key is not specified, the current target id is used.
param_groups	No	List (param_group)	OPTIONAL list of type `param_group`. The parameters specified in each `param_group` will be used to override (see Override Process) the parameters of the corresponding `param_group` in the default manifest where the ID matches If no parameters are being overriden, `param_groups` does not need to be specified. The `interpreter` Dict is never required in the Input Manifest. Environment parameters can be unset by assigning `''` as the value for a param. A `param_group` can have either `params` or `params_from_file` but not both.

4.1.4. pdi Syntax

We can use this section either image or dockerfile(not both).

Key	Required?	Value Type	Notes
image	No	Scalar (string)	The name of the container image for the PDI. This can accept values including the registry e.g: `registry.gear.ge.com/pwr-sdp/pdi-scada` OR local images that have not been pulled from a registry e.g: `pdi-scada`
tag	No	Scalar (string)	Tag name of the docker image .

Key	Required?	Value Type	Notes
dockerfile	No	Scalar (string)	Specifies a path to a dockerfile on the host. See Example Manifest using dockerfile for an example of this. When using dockerfile, image parameter must not be used.
build-args	No	Scalar (string)	Specifies the build args for the dockerfile. This is optional. See Example Manifest using dockerfile for an example of this.
remote_image	No	Boolean	Default: `false` Setting remote_image to true will force a pull of the image from a remote repository. If the image is not available then bsfdeploy will fail. If `remote_image` is set to false, bsfdeploy will attempt to pull the image from a remote repository and fallback to a local image of the same name.
local_image	No	Boolean	Default: `false` Setting local_image to true will prevent pull of the docker image (only the local image will be used). If the local image is not available then bsfdeploy will fail.

4.1.5. param_group Syntax

Key	Required?	Value Type	Notes
id	Yes	Scalar (string)	MUST be unique and match the `id` of that `param_group` in the `default_manifest.yaml` file so that the override process knows which `param_group` to insert the value into.
params	No	Dict	Dictionary of parameters (the top-level params MUST be of type Dict). Nested params MAY be type Dict, List or Scalar. An empty parameter dictionary can be specified, allowing users to provide their own parameters using either `params:` or `params: None`
params_from_file	No	Scalar (string)	Specifies a path to a file on the host file system containing the parameters to be overriden. The file type needs to match the interpreter type for the `param_group` as specified in the Default Manifest. For example, if the interpreter type is `json`, the file specified should be a `json` file. See Example Input Manifest for an example of this.

4.1.6. Example Input Manifest

```yaml
id: 'solution-a'
version: '1'
api_version: '5.0'
targets:
  - id: 'target-A'
    products:
      - id: 'product-X'
        pdi:
          image: 'registry.gear.ge.com/pwr-x/product-x-deploy'
          tag: '1.0.0'
        local_dir_mount_path: '/path/to/local/dir'
        param_groups:
          - id: 'group_a'
            params:
              param_a: 'override_value'
```

Example Manifest using `params_from_file`

```yaml
id: 'solution-a'
version: '1'
api_version: '5.0'
targets:
  - id: 'target-A'
    products:
      - id: 'product-X'
        pdi:
          image: 'registry.gear.ge.com/pwr-x/product-x-deploy'
          tag: '1.0.0'
        local_dir_mount_path: '/path/to/local/dir'
        param_groups:
          - id: 'group_a'
            params_from_file: '/path/to/param/file'
```

4.2. Example Manifest using dockerfile

```yaml
id: 'solution-a'
version: '1'
api_version: '5.0'
targets:
  - id: 'target-A'
    products:
      - id: 'product-X'
        pdi:
          dockerfile: "/path/to/dockerfile"
          build-args: # optional
           ARG_ONE: "example_arg"
           ARG_TWO: "example_arg"
           ARG_THREE: "example_arg"
          tag: "image:tag" # optional
        local_dir_mount_path: '/path/to/local/dir'
        param_groups:
          - id: 'group_a'
            params:
              param_a: 'override_value'
```

4.3. Override Process

4.3.1. Inputs and Outputs of the Override Process

4.3.1.1. Default Manifest (Input)

The `Default Manifest` is a YAML file that is built into each PDI. It contains the default parameters to be used by the deployment automation. The parameters themselves are arranged into groups of parameters known as `param_groups` and their default values are specified in the `params` key of each `param_group`. The `params_from_file` key can be used to specify default parameters from a separate file, rather than putting them in the Default Manifest directly. When `params_from_file` is used, the parameters specified in the file are loaded into the `params` key at runtime of the PDI.

4.3.1.2. Input Manifest (Input)

When a PDI is run with `--action` equal to `plan` or `apply`, an Top Level Syntax YAML file MUST be supplied (this is done via a volume mount to /data/input/manifest.yaml). This file contains override parameters for each product within a target, to be used instead of those defined in the Default Manifest. These parameters are also arranged into `param_groups` as per the Default Manifest.

4.3.1.3. Rendered Manifest (Output)

The override process is executed dynamically on the corresponding `params` section from the default manifest and the input manifest. The result of the process is called the Rendered Manifest. It can be displayed using the `plan` action with the verbose flag.

4.3.2. Override Behaviour

- The override process loops through each `param_group` specified in the Input Manifest, and searches for a corresponding `param_group` (with matching `id` value) in the Default Manifest.
- If a matching group is found, it then loops through each of the parameters in the `params` dictionary of the `param_group`, and looks for a parameter of the same name in the matching default `param_group`.
  - If a parameter exists in the Default Manifest but not in the Input Manifest, the value in the Default Manifest will be used in the Rendered Manifest.
  - If a parameter exists in the Input Manifest but not in the Default Manifest, the `key: value` pair will be added to the `params` dictionary in the Rendered Manifest.
- Assuming a matching parameter is found in the default `param_group`, the value type (e.g. `scalar`, `list` or `dict`) of the paramater is then compared, and it must be identical to the value type of the parameter of the same name in the default `param_group`.
- Assuming the type is correct, the override of the parameter is processed. The override behaviour depends on the value type of parameter:

4.3.2.1. Scalar

If the value is of type `scalar`, the value of the parameter that gets rendered will simply be the value of the override parameter specified in the Input Manifest.

4.3.2.2. List

If the value is of type `list`, the behaviour is similar to that of a `scalar`: the list that gets rendered will be the same list that is specified in the Input Manifest.

> When overriding lists, the list in the Input Manifest is not merged with the list in the Default Manifest, any list items specified in the Default Manifest will be lost. This can be avoided by using the indexed notation.

4.3.2.3. List item

Using an indexed notation `xxx[n]:`, you can do the override of an individual list item. ex: In the default manifest:

```
servers:
  - hostname: 'name1'
    ip: '10.0.0.1'
  - hostname: 'name2'
    ip: '10.0.0.2'
```

In the input manifest:

```
servers[0]:
  hostname: 'newname'
```

Then the rendered manifest will be:

```
servers:
  - hostname: 'newname'
    ip: '10.0.0.1'
  - hostname: 'name2'
    ip: '10.0.0.2'
```

- As shown in the example, list indexes start at 0.
- If index doesn't correspond to an existing list index of default manifest, it will be refused (here servers[2]:... will be refused).
- To add or remove an item from the list defined in the default manifest, you need to respecify the entire list in the input manifest, modified with the added or removed items.

4.3.2.4. Dictionary

If the value is a dictionary, each `key: value` pair in the nested dictionary will be processed recursively based on it's value type (`scalar`, `list`, `dict`), following the same rules as above.

4.3.3. Example 1 with Scalar Value

- The Default Manifest of the ProductA:

```
product_id: 'ProductA'
run_cmd: 'cmdA'
param_groups:
  - id: 'GroupA'
    interpreter:
      type: 'yaml'
      outfile: 'src/tests/tmp/values.yml'
    params:
      paramAAX: 'defaultAAX'
      paramAAY: 'defaultAAY'
      paramAAZ: 'defaultAAZ'
```

- The Input Manifest of the SolutionA:

```yaml
id: 'SolutionA'
targets:
  - id: 'TargetA'
    products:
      - id: 'ProductA'
        pdi:
          image: 'productA-Deploy'
          tag: '1.0.0'
        param_groups:
          - id: 'GroupA'
            params:
              paramAAX: 'overrideAAAX'
              paramAAY: 'overrideAAAY'
```

- The rendered params section that results from the override process on TargetA, ProductA:

```yaml
params:
  paramAAX: 'overrideAAAX'
  paramAAY: 'overrideAAAY'
  paramAAZ: 'defaultAAZ'
```

4.3.4. Example 2 with List, Dictionary, and missing parameters

- The Default Manifest of the ProductB:

```yaml
product_id: 'ProductB'
run_cmd: 'cmdA '
param_groups:
  - id: 'GroupA'
    interpreter:
      type: 'yaml'
      outfile: '/data/values.yml'
    params:
      paramAAX: 'defaultAAX'
      paramAAY: 'defaultAAY'
      paramAAZ: 'defaultAAZ'
      dictAAX:
        dictAAX_paramX: 'defaultAAXX'
        dictAAX_paramY: 'defaultAAXY'
      listAAX:
        - listAAX_paramX: 'defaultAAXX'
        - listAAX_paramY: 'defaultAAXY'
```

- The Input Manifest for SolutionB:

```
id: 'SolutionB'
targets:
  - id: 'TargetB'
    products:
      - id: 'ProductB'
        pdi:
          image: 'productA-Deploy'
          tag: '1.0.0'
        param_groups:
          - id: 'GroupA'
            params:
              paramAAW: 'newparamsAAAW'
              paramAAX: 'overrideAAAX'
              paramAAY: 'overrideAAAY'
              dictAAX:
                dictAAX_paramX: 'overrideAAXX'
                dictAAX_paramZ: 'new_paramsAAXZ'
              listAAX:
                - newlistAAX_paramX: 'newparamAAXX'
```

- The rendered params section that results from the override process on TargetB, ProductB:

```
params:
  paramAAW: 'newparamsAAAW'
  paramAAX: 'overrideAAAX'
  paramAAY: 'overrideAAAY'
  paramAAZ: 'defaultAAZ'
  dictAAX:
    dictAAX_paramX: 'overrideAAXX'
    dictAAX_paramY: 'defaultAAXY'
    dictAAX_paramZ: 'new_paramsAAXZ'
  listAAX:
    - newlistAAX_paramX: 'newparamAAXX'
```

4.4. Templating Process

The Input Manifest may be used to describe a complete solution, including multiple targets and products. Templating has been introduced in the Solution Engine to leverage the dependency between parameters of different product deployments and to create reusable targets description.

4.4.1. Jinja2 Settings

The jinja2 templating engine is used to process the template and generate the final input manifest. Please refer to the following link for the jinja2 syntax: http://jinja.pocoo.org/docs/2.10/templates/ The jinja2 environment created during the process is using the default jinja2 setup as defined in the class jinja2.Environment on http://jinja.pocoo.org/docs/2.10/api/ except :

- the `loader` parameter is set to jinja2.FileSystemLoader. Template are loaded as file. The base folder for the template is set to the folder of the input manifest

- the `trim_blocks` parameter is set to True, to remove the first new line after a block
- the `lstrip_blocks` parameter is set to True, to strip leading space and tabs
- the `undefined` parameters is set to `jinja2.StrictUndefined`, to raise an exception if a variable is not defined in the template.

4.4.1.1. Jinja2 env Filter

It is possible to use host environment variables inside the template. This is done using the syntax:

```
{{ env["ENV_VAR_NAME"] }}
```

The environment variable will be resolved into a string at run time during template processing.

4.4.1.2. Jinja2 gethostbyname Filter

The `gethostname` filter allows hostname resolution to IP address, during the processing of the template. The hostname resolution is performed by the host bsfdeploy is running on.

For example if the IP address of a target called `example.local` is required, then, in the Input Manifest, the following syntax can be used:

```
params:
  target_ip: {{ example.local | gethostbyname }}
```

4.4.2. Processing

The Input Manifest file given as parameter to the solution engine is processed by the jinja2 engine. The outcome is stored on a temporary YAML file. This file is validated against the YAML format and against the Input Manifest format. If this is a regular YAML file, it won't be modified. If no action is specified by the Input Manifest, it will be generated for validation.

4.4.2.1. Example 1 - All in One

- In this example the initialization of the global parameters, the definition of the jinja2 macro and the template itself is in the same file

```
{% set globals_parameters %}
            param_g1: vg_1
            param_g2: vg_2
{%- endset %}
{% set artifact_repository  %}
            repo_g1: https://nexus/repository/bsf-generic-virtual
{%- endset %}
{%  set docker_repo %}
https://dockerhub/repository/bsf
{%- endset %}
{% set dest_folder  %}
/home/deploy
{%- endset %}

{# target templates #}
{% macro target_template(name,host_ip) %}
    products:
    - id: 'Product_A'
      pdi:
           image: {{ docker_repo }}/test_product_a
           tag: '1.0.0'
      param_groups:
        - id: 'Product_A_Group_A'
          params:
            hostname: {{ name }}
            ip: {{ host_ip }}
{{ globals_parameters }}
{{ artifact_repository }}
            param_local_1: pl_1
    - id: 'Product_B'
      pdi:
           image: {{ docker_repo }}/test_product_b
           tag: '1.0.0'
      param_groups:
        - id: 'Product_B_Group_A'
          params:
            hostname: {{ name }}
            ip: {{ host_ip }}
            param_local_2: pl_2
{%- endmacro %}
{# Input Manifest #}
# Input Manifest
id: 'All_in_one_template_example'
version: '1.0'
APIversion: '2.0.0'
targets:
  - id: 'emsa'
{{ target_template('MYEMSA','10.0.0.1') }}
  - id: 'emsb'
{{ target_template('MYEMSB','10.0.0.2') }}
```

- The generated Input Manifest will be:

```yaml
# Input Manifest
id: 'All_in_one_template_example'
version: '1.0'
APIversion: '2.0.0'
targets:
  - id: 'emsa'
    products:
    - id: 'Product_A'
      pdi:
           image: https://dockerhub/repository/bsf/test_product_a
           tag: '1.0.0'
      param_groups:
        - id: 'Product_A_Group_A'
          params:
            hostname: MYEMSA
            ip: 10.0.0.1
            param_g1: vg_1
            param_g2: vg_2
            repo_g1: https://nexus/repository/bsf-generic-virtual
            param_local_1: pl_1
    - id: 'Product_B'
      pdi:
           image: https://dockerhub/repository/bsf/test_product_b
           tag: '1.0.0'
      param_groups:
        - id: 'Product_B_Group_A'
          params:
            hostname: MYEMSA
            ip: 10.0.0.1
            param_local_2: pl_2
  - id: 'emsb'
    products:
    - id: 'Product_A'
      pdi:
           image: https://dockerhub/repository/bsf/test_product_a
           tag: '1.0.0'
      param_groups:
        - id: 'Product_A_Group_A'
          params:
            hostname: MYEMSB
            ip: 10.0.0.2
            param_g1: vg_1
            param_g2: vg_2
            repo_g1: https://nexus/repository/bsf-generic-virtual
            param_local_1: pl_1
    - id: 'Product_B'
      pdi:
           image: https://dockerhub/repository/bsf/test_product_b
           tag: '1.0.0'
      param_groups:
        - id: 'Product_B_Group_A'
          params:
            hostname: MYEMSB
```

```
            ip: 10.0.0.2
            param_local_2: pl_2
```

4.4.2.2. Example 2 - Using Template Inheritance and Import

- In this example, global parameters, macro and template are split in 3 files. The global parameters file inherit the template (`extends`) and the template import the macro (`import`). The files are located in the same folder. the global parameters file will be given as --input-manifest argument to the solution engine.
- The global parameters file:

```
{% extends 'template_dual_target.j2' %}
{% set globals_parameters %}
            param_g1: vg_1
            param_g2: vg_2
{%- endset %}
{% set artifact_repository  %}
            repo_g1: https://nexus/repository/bsf-generic-virtual
{%- endset %}
{%  set docker_repo %}
https://dockerhub/repository/bsf
{%- endset %}
{% set dest_folder  %}
/home/deploy
{%- endset %}
```

- The macro file (template-macro.j2):

```
{# target templates #}
{% macro target_template(name,host_ip,globals_parameters,artifact_repository,
   docker_repo) %}
    products:
    - id: 'Product_A'
      pdi:
           image: {{ docker_repo }}/test_product_a
           tag: '1.0.0'
      param_groups:
        - id: 'Product_A_Group_A'
          params:
            hostname: {{ name }}
            ip: {{ host_ip }}
{{ globals_parameters }}
{{ artifact_repository }}
            param_local_1: pl_1
    - id: 'Product_B'
      pdi:
           image: {{ docker_repo }}/test_product_b
           tag: '1.0.0'
      param_groups:
        - id: 'Product_B_Group_A'
          params:
            hostname: {{ name }}
            ip: {{ host_ip }}
            param_local_2: pl_2
{%- endmacro %}
```

- The template file (template_dual_target.j2):

```
{% from 'template-macro.j2' import target_template %}
{# Input Manifest #}
# Input Manifest
id: 'Dual_target'
version: '1.0'
APIversion: '2.0.0'
targets:
  - id: 'emsa'
{{ target_template('MYEMSA','10.0.0.1',globals_parameters,artifact_repository, docker_repo) }}
  - id: 'emsb'
{{ target_template('MYEMSB','10.0.0.2',globals_parameters,artifact_repository, docker_repo) }}
```

- The generated Input Manifest will be identical to the one of the previous example.

4.5. Secret Management

Some of the parameters in the input manifest cannot be in clear text, like credentials. The deployment engine supports the decryption of parameters in memory before running the interpreter. Secret parameters will be logged and stored in the state file in their encrypted format.

The encryption that is used by the deployment engine is `RSA+OAEP` (Optimal Asymmetric Encryption

Padding)

Encryption is done using a public key generated outside the deployment engine. Decryption is done by the corresponding private key within the deployment engine. The public key/private key pair should be unique for the solution.

By convention, parameters can be marked as secrets to be decrypted when they are specified in the Default Manifest by adding the '__' (2 underscore characters) prefix to the param name. This prefix is removed from the parameter name during encryption.

Parameters not marked as secrets in the Default Manifest (without '__' prefix), can also be overridden by secret parameters by simply adding the prefix to the name of the parameter in the Input Manifest.

Similarly, parameters marked as secrets in the Default Manifest can be overridden by plain text values by removing the prefix from the parameter name in the Input Manifest.

The value of a secret parameter must be a base64 encoded string.

4.5.1. Generation of the Keys

```
openssl genrsa -out solution_solution_a.pem 2048
openssl rsa -in solution_solution_a.pem -outform PEM -pubout -out public.pem
```

Once generated, the private key should be stored in a safe environment.

4.5.2. Encryption of Parameters

Use the public key to encrypt the secret parameters as a string

```
echo -n my_secret_password | openssl rsautl -encrypt -oaep -pubin \
     -inkey public.pem |base64 -w 0
```

4.5.3. Decryption of the Parameters

The decryption of the secret parameters is done in memory just before calling the interpreters, using the private key.

The path to the key file can be specified by the -k (--encryption_key) argument. By default, the PDI is looking for the private key at `/data/secret/solution_<solution_id>.pem`. The bsfdeploy client will mount the key into the `/data/secret` volume.

The Deployment Engine decrypts parameters that are prefixed with '__' (in either the Input or Default manifests) after the override process, but before running the interpreters. It is expected that the secret parameters are defined but not set in the default manifest. Secret parameters can be defined in the params section of a param_group but also in a params_from_file when supported by the interpreter.

4.5.4. Example

- Default Manifest

```yaml
product_id: 'product-X'
run_cmd: './run.sh'
param_groups:
  - id: 'group-a'
    interpreter:
      type: '<interpreter type>'
      <...type specific mandatory parameters...>
    params:
      param_a: '<default value of param_a>'
      param_b: '<default value of param_b>'
      __secret_param: ''
```

- Input Manifest

```yaml
id: 'solution_a'
version: '1'
api_version: '1'
targets:
  - id: 'target-A'
    products:
      - id: 'product-X'
        pdi:
          image: 'registry.gear.ge.com/pwr-x/product-x-deploy'
          tag: '1.0.0'
        param_groups:
          - id: 'group_a'
            params:
              __secret_param: 's/XX+Ev4/qNknTO5EqDiL4VOp1PTYJmwQDNL'
```

The `param_groups` process by the interpreter in memory will be:

```yaml
params:
    param_a: '<default value of param_a>'
    param_b: '<default value of param_b>'
    secret_param: my_secret_password
```